IDAT7215
Computer Programming for Product Development and Applications

Lecture 2-3: Python Libraires: SciPy

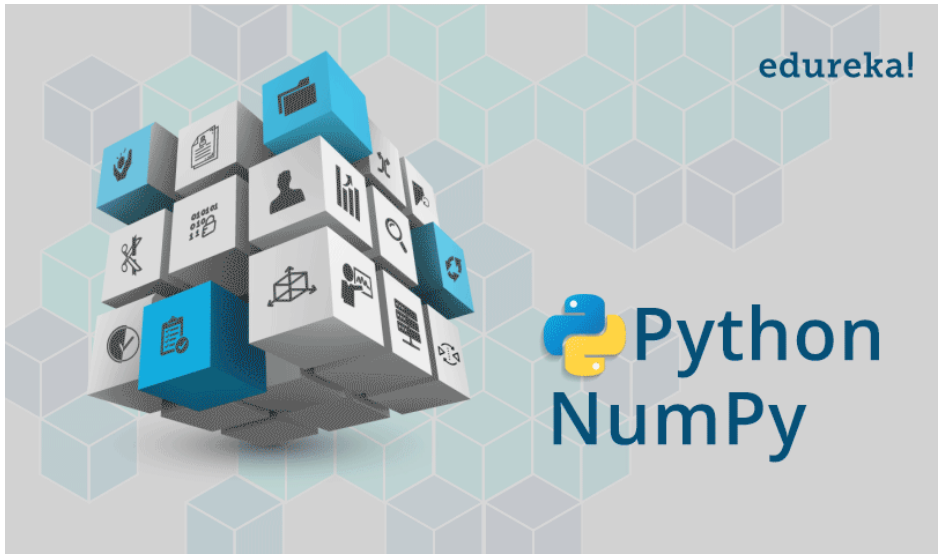
Dr. Zulfiqar Ali

Outline

- What is Python SciPy
- NumPy vs SciPy
- Sub-Packages in SciPy
- Basic Functions
- Special Functions
- Integration Functions
- Fourier Transformations
- Linear Algebra
- Optimization Function
- Interpolation Functions
- Curve Fitting Function

What is Python SciPy

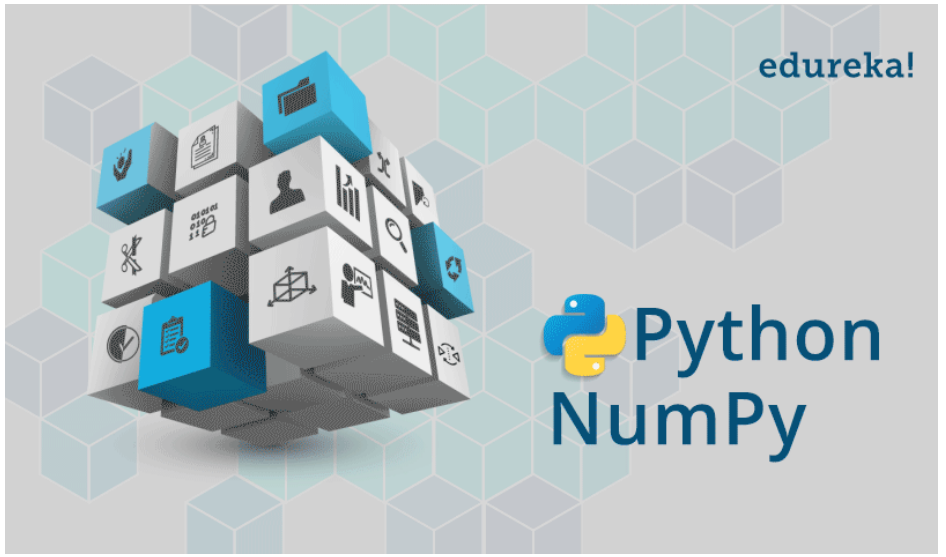
- SciPy is a python library used to solve scientific and mathematical problems
- Built on NumPy
- Allows manipulation and visualizing



SciPy

NumPy vs SciPy

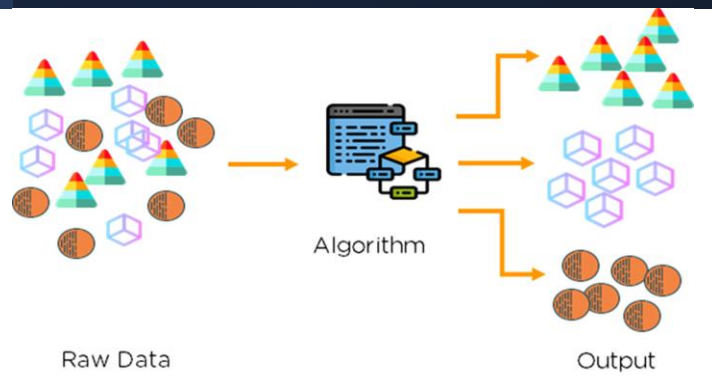
- NumPy and SciPy used for mathematical and numerical analysis
- NumPy contains array data and basic operations such as sorting indexing etc.
- SciPy consists of all the numerical code.
- SciPy contains fully-featured versions of mathematical and scientific functions.



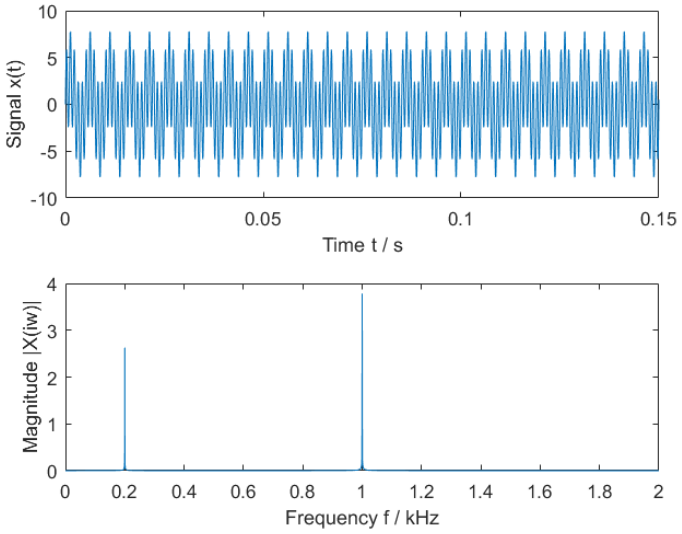
SciPy

Sub-Packages in SciPy

Name	Description
cluster	Clustering algorithms
constants	Physical and mathematical constants
fftpack	Fast Fourier Transform routines
integrate	Integration and ordinary differential equation solvers
interpolate	Interpolation and smoothing splines
io	Input and Output
linalg	Linear Algebra
ndimage	N-dimensional image processing
odr	Orthogonal distance regression
optimize	Optimization and root finding
signal	Signal Processing
sparse	Sparse matrices and associated routines
spatial	Spatial data structures and algorithms
special	Special functions
stats	Statistical distributions and functions

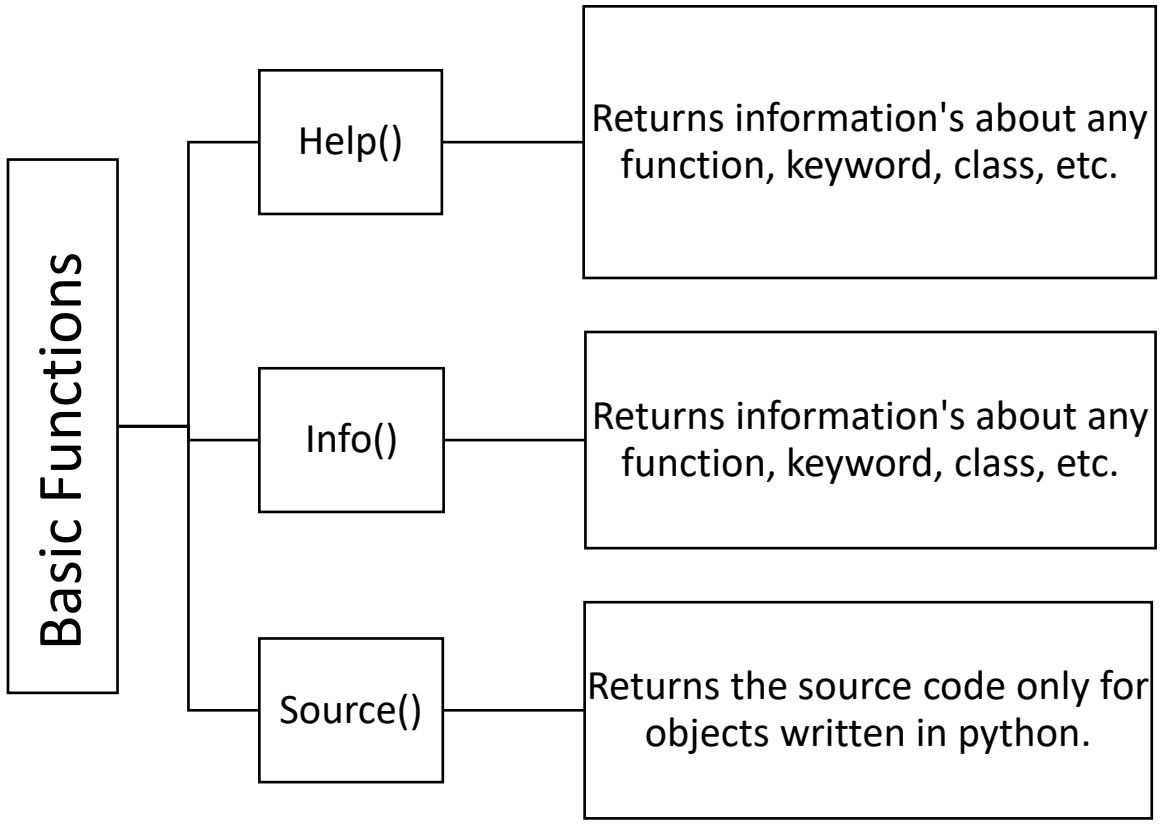


Clustering of Raw data



Fourier Transform

Basic Functions




```
from scipy import cluster
```

```
help(cluster)
```

```
Help on package scipy.cluster in scipy:

NAME
    scipy.cluster

DESCRIPTION
    ========================================
    Clustering package (:mod:`scipy.cluster`)
    ========================================

    .. currentmodule:: scipy.cluster

    :mod:`scipy.cluster.vq`

    Clustering algorithms are useful in information theory, target detection,
    communications, compression, and other areas. The `vq` module only
    supports vector quantization and the k-means algorithms.

    :mod:`scipy.cluster.hierarchy`

    The `hierarchy` module provides functions for hierarchical and
    agglomerative clustering.  Its features include generating hierarchical
    clusters from distance matrices,
    calculating statistics on clusters, cutting linkages
    to generate flat clusters, and visualizing clusters with dendrograms.

PACKAGE CONTENTS
    _hierarchy
    _optimal_leaf_ordering
    _vq
    hierarchy
    setup
    tests (package)
    vq
```

Jupyter Notebook

Basic Functions

```
from scipy import cluster
```

```
help(cluster)
```

```
Help on package scipy.cluster in scipy:

NAME
    scipy.cluster

DESCRIPTION
    ========================================
    Clustering package (:mod:`scipy.cluster`)
    ========================================

    .. currentmodule:: scipy.cluster

    :mod:`scipy.cluster.vq`

    Clustering algorithms are useful in information the
    communications, compression, and other areas. The `
```

```
import scipy
scipy.info(cluster)
```

```
========================================
Clustering package (:mod:`scipy.cluster`)
========================================

.. currentmodule:: scipy.cluster

:mod:`scipy.cluster.vq`

Clustering algorithms are useful in information
communications, compression, and other areas. T
supports vector quantization and the k-means al

:mod:`scipy.cluster.hierarchy`
```

```
import scipy
scipy.source(cluster)
```

```
In file: D:\C\User\anaconda3\lib\site-packages\scipy\

"""
========================================
Clustering package (:mod:`scipy.cluster`)
========================================

.. currentmodule:: scipy.cluster

:mod:`scipy.cluster.vq`

Clustering algorithms are useful in information theor
communications, compression, and other areas. The `vq
supports vector quantization and the k-means algorith

:mod:`scipy.cluster.hierarchy`
```

Jupyter Notebook

Special Functions

- Functions available for Mathematical Physics.
- Some of these functions included gamma, beta, hypergeometric, parabolic cylinder etc.

Exponential Functions

$a = 10^2$ →

```
from scipy import special
a = special.exp10(2)
print(a)
```

```
100.0
```

$b = 2^3$ →

```
from scipy import special
b = special.exp2(3)
print(b)
```

```
8.0
```

Special Functions

- Functions available for Mathematical Physics.
- Some of these functions included gamma, beta, hypergeometric, parabolic cylinder etc.

Trigonometric Functions

$c = \sin(90°)$ →

```
from scipy import special
c = special.sindg(90)
print(c)
```

```
1.0
```

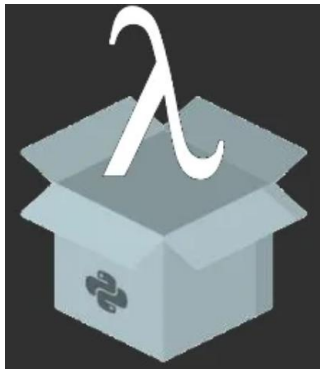
$d = \cos(90°)$ →

```
from scipy import special
c = special.cosdg(90)
print(c)
```

```
-0.0
```

Lambda Functions

- Lambda functions or ***anonymous functions***.
- ***One-time use***: Also known as throw-away function as they are needed just once.
- ***I/O of other Functions***: They are also passed as inputs or returned as outputs of other higher order functions.
- ***Reduce code size***: The body of the lambda function is written in a single line



$$f = x - y$$

```
# General function
def minus(x,y):
    return x-y
print(minus(9,4))
```

```
5
```

```
# Lambda function or anonymous functions
minus= lambda x,y: x-y
print(minus(9,4))
```

```
5
```

Integration Functions

- Integration deal with adding slices to determine the whole.
- Integration can be used to find displacement, area etc.

General Integration

The quad function calculates the integral of a function which has one variable in SciPy.

$$i = \int_0^1 10^x \, dx$$

```
from scipy import integrate

i=integrate.quad(lambda x:special.exp10(x),0,1)

print(i)
```

```
(3.9086503371292665, 4.3394735994897923e-14)
```

Integration Functions

- Integration deal with adding slices to determine the whole.
- Integration can be used to find displacement, area etc.

Double Integration

The dblquad function calculates the integral of a function which has two variables in SciPy.

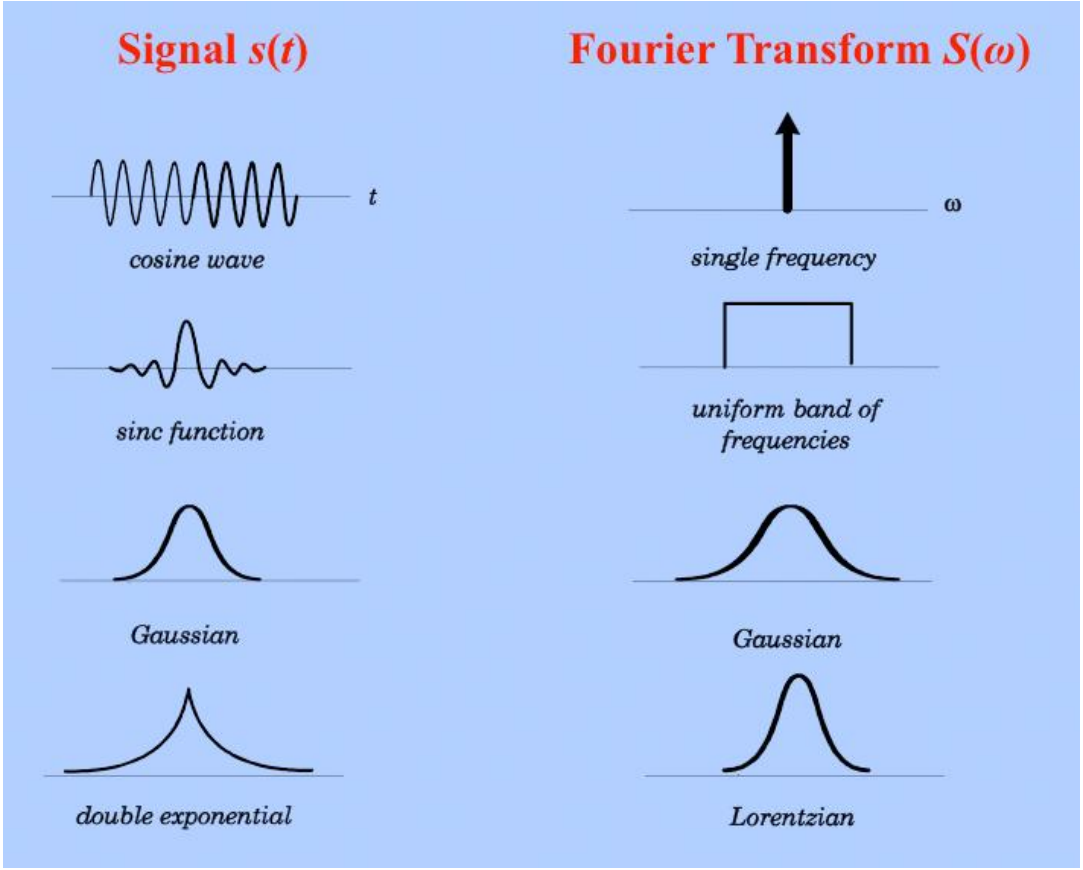
$$e = \int_{x=0}^{x=2} \int_{y=0}^{y=1} xy^2 dydx$$

```
# Double Integration
from scipy import integrate
f = lambda x, y: x*y**2
e = integrate.dblquad(f,0,2,0,1)
print(e)
```

```
(1.3333333333333335, 2.210813483580884e-14)
```

Fourier Transformation

- Fourier analysis is a method that deals with expressing a function as a sum of periodic components and recovering the signal from those components.
- The fft and ifft functions can be used to return the discrete Fourier Transform of a real or complex sequence.



Fourier Transformation

- Fourier analysis is a method that deals with expressing a function as a sum of periodic components and recovering the signal from those components.
- The fft and ifft functions can be used to return the discrete Fourier Transform of a real or complex sequence.

Fourier Transform

$x = [1\ 2\ 3\ 4]$

```
from scipy.fftpack import fft,ifft
import numpy as np
x=np.array([1,2,3,4])
y=fft(x)
print(y)
```

```
[10.-0.j -2.+2.j -2.-0.j -2.-2.j]
```

```
from scipy.fftpack import fft,ifft
x=np.array([1,2,3,4])
y=fft(x)
print(y)
xx=ifft(y)
print(xx)
```

```
[10.-0.j -2.+2.j -2.-0.j -2.-2.j]
[1.+0.j 2.+0.j 3.-0.j 4.+0.j]
```

Linear Algebra

- SciPy is built on ATLAS LAPACK and BLAS libraries and is extremely fast in solving problems related to linear algebra
- ***Inverse of a Matrix***: Inverse of a matrix A is the matrix B such that $AB = I$ where I is identity matrix and $B = A^{-1}$.

Inverse of Matrix

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

```
from scipy import linalg
import numpy as np
A = np.array([[1,2],[3,4]])
B = linalg.inv(A)
print('B = ', B)
```

```
B =  [[-2.   1. ]
 [ 1.5 -0.5]]
```

Optimization

- SciPy have build in functions to solve the optimization problem.

$$Minimize\ f(x) = (x-3)^2$$

```
from scipy.optimize import minimize

def f(x):
    return (x-3)**2

res = minimize(f,2)
res.x
```

```
array([2.99999999])
```

Minimization

```
from scipy.optimize import minimize

def f(x):
    return (x-3)**2

res = minimize(f,2)
res
```

Initial Guess

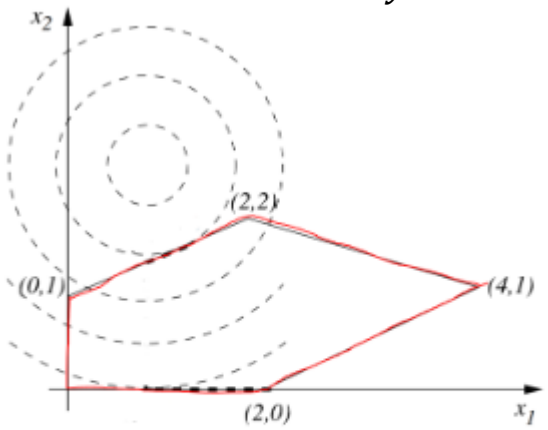
```
      fun: 5.551437397369767e-17
 hess_inv: array([[0.5]])
      jac: array([-4.3254289e-13])
  message: 'Optimization terminated successfully.'
     nfev: 6
      nit: 2
     njev: 3
   status: 0
  success: True
        x: array([2.99999999])
```

Optimization

$Minimize\ f(x,y) = (x-1)^2 + (y-2.5)^2$

$Subject\ to$:

$$x - 2y + 2 \geq 0$$
$$-x - 2y + 6 \geq 0$$
$$-x + 2y + 2 \geq 0$$
$$x \geq 0$$
$$y \geq 0$$



- 2D function takes in vector x
- Constraints must be specified as $g_i(x) \geq 0$
- Bounds specified as rectangular

Minimization

```
from scipy.optimize import minimize


f = lambda x: (x[0] - 1)**2 + (x[1] - 2.5)**2
cons = ({'type': 'ineq', 'fun': lambda x:  x[0] - 2 * x[1] + 2},
        {'type': 'ineq', 'fun': lambda x: -x[0] - 2 * x[1] + 6},
        {'type': 'ineq', 'fun': lambda x: -x[0] + 2 * x[1] + 2})
bnds = ((0, None), (0, None))
res = minimize(f, (2, 0), bounds=bnds, constraints=cons)

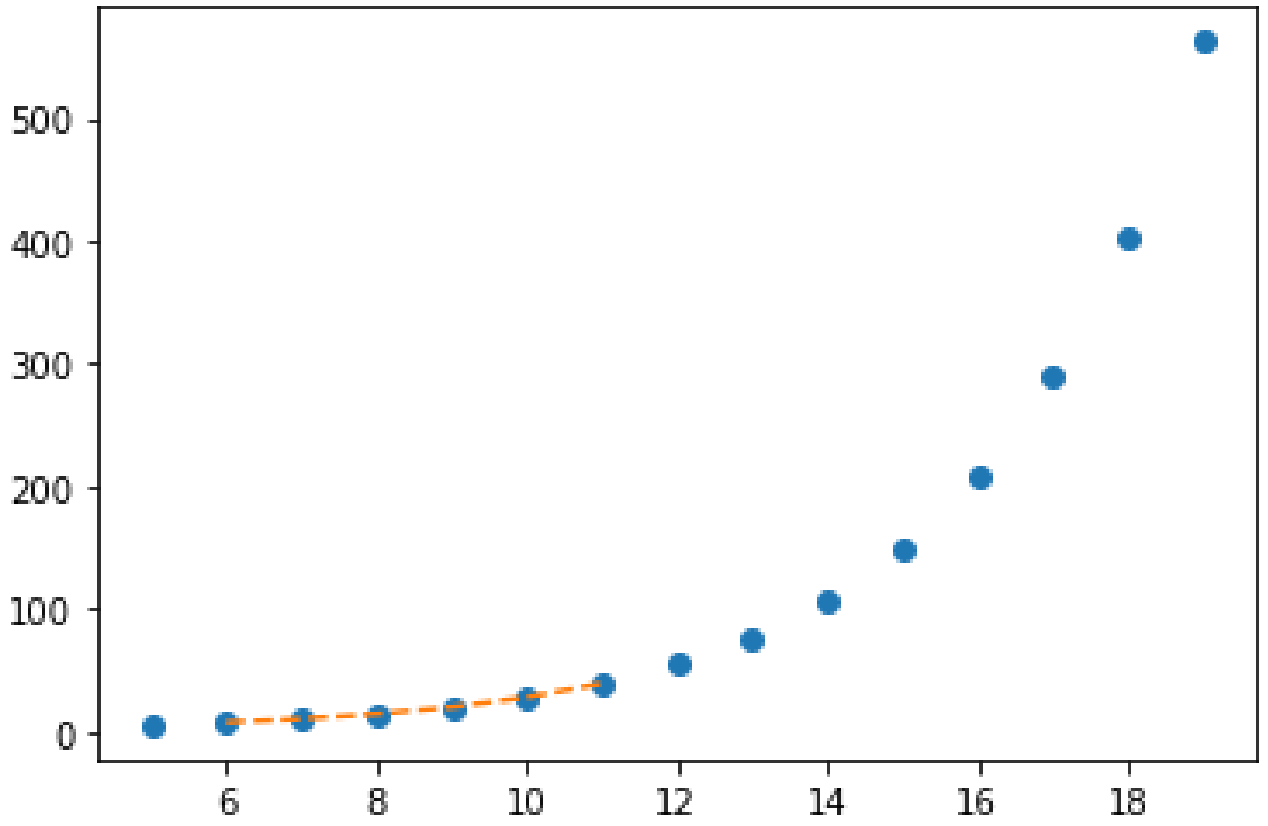
res.x
```

```
array([1.4, 1.7])
```

Initial Guess

Interpolation Functions

- **Interpolation** refers to constructing new data points within a set of known data points.
- The scipy.interpolate consists of spline functions and classes, one-dimensional and multi-dimensional (univariate and multivariate) interpolation classes, etc.

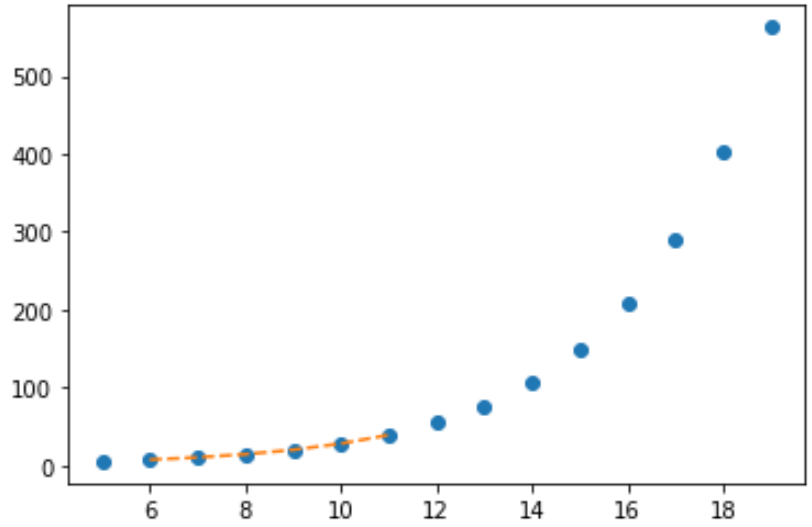


Interpolation Functions

- **Interpolation** refers to constructing new data points within a set of known data points.
- The scipy.interpolate consists of spline functions and classes, one-dimensional and multi-dimensional (univariate and multivariate) interpolation classes, etc.

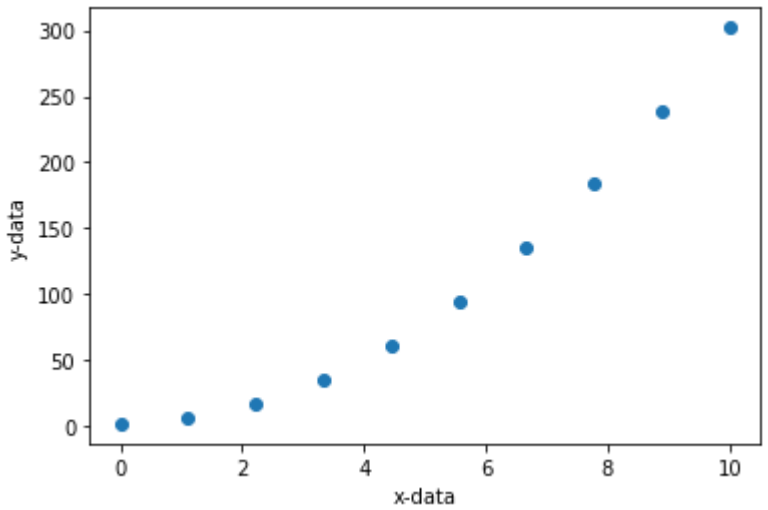
Interpolation

```
import matplotlib.pyplot as plt
from scipy import interpolate
x = np.arange(5,20)
y = np.exp(x/3.0)
f=interpolate.interp1d(x,y)
x1=np.arange(6,12)
y1=f(x1)  # Use interpolation function reurned by int
plt.plot(x,y,'o', x1, y1, '--')
plt.show;
```



Curve Fitting

- Curve fitting is the process of constructing a curve, or mathematical function, that has the best fit to a series of data points



Curve Fitting

$$y = ax^2 + b$$

$Find\ a\ and\ b\ if\ x\ and\ y\ is\ given$

```
# Want to fit the data to curve y = ax^2 +b. The main goal here is determine the values of a and b.
from scipy.optimize import curve_fit

x_data = np.linspace(0, 10, 10)
y_data = 3*x_data**2 + 2

def func(x, a, b):
    return a*x**2 + b
popt, pcov = curve_fit(func, x_data, y_data, p0=(1,1))
popt
```

```
array([3., 2.])
```

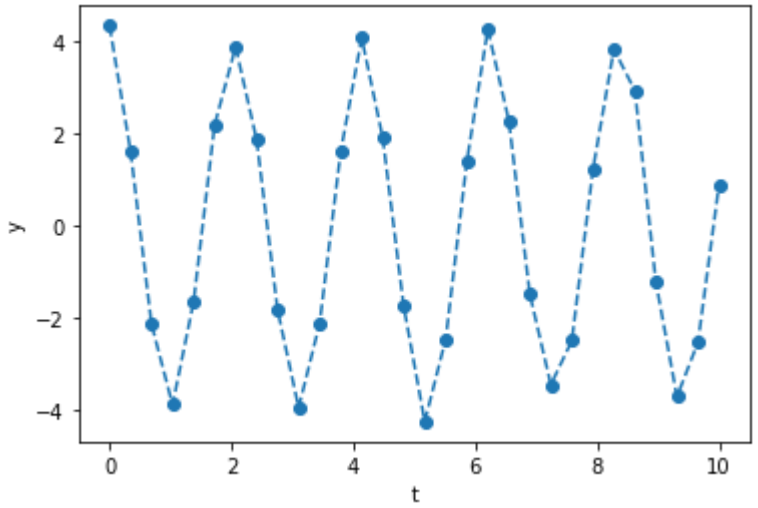
```
pcov
```

```
array([[ 4.13297331e-34, -1.45419433e-32],
       [-1.45419433e-32,  9.65872308e-31]])
```

Initial Guess

Curve Fitting

- The equation of spring motion is $y(t) = Acos(wt + \emptyset)$
- Want to find the natural frequency of oscillation w for the spring.



Curve Fitting

$$y(t) = Acos(wt + \emptyset)$$

$$Find\ A, w\ and\ \emptyset\ \ if\ t\ and\ y\ is\ given$$

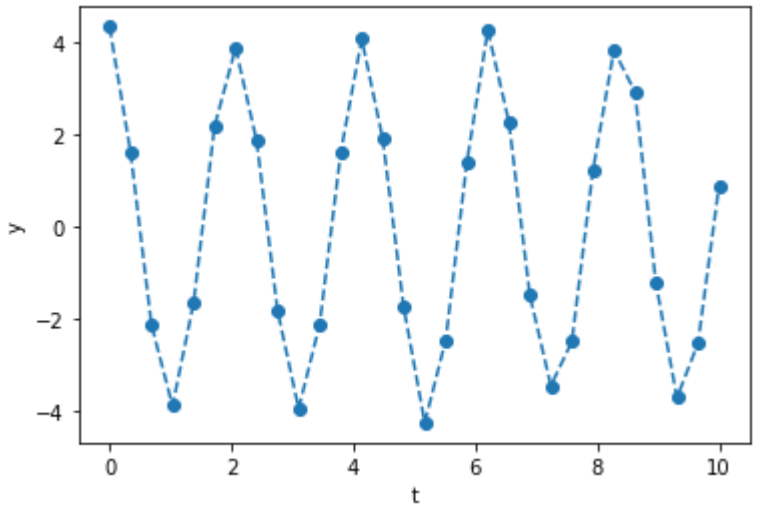
$$w = 2\pi f, \qquad f = \frac{1}{T}, \qquad T \approx 2$$

The initial good guess

- $w = 2\pi f = \pi$
- $A = 4$
- $\emptyset = 0$

Curve Fitting

- Curve fitting is the process of constructing a curve, or mathematical function, that has the best fit to a series of data points



Curve Fitting

```
t_data = np.array([ 0.   ,  0.34482759,  0.68965517,  1.03448276,  1.37931034,
        1.72413793,  2.06896552,  2.4137931 ,  2.75862069,  3.10344828,
        3.44827586,  3.79310345,  4.13793103,  4.48275862,  4.82758621,
        5.17241379,  5.51724138,  5.86206897,  6.20689655,  6.55172414,
        6.89655172,  7.24137931,  7.5862069 ,  7.93103448,  8.27586207,
        8.62068966,  8.96551724,  9.31034483,  9.65517241, 10.        ])
y_data = np.array([ 4.3303953 ,  1.61137995, -2.15418696, -3.90137249, -1.67259042,
        2.16884383,  3.86635998,  1.85194506, -1.8489224 , -3.96560495,
       -2.13385255,  1.59425817,  4.06145238,  1.89300594, -1.76870297,
       -4.26791226, -2.46874133,  1.37019912,  4.24945607,  2.27038039,
       -1.50299303, -3.46774049, -2.50845488,  1.20022052,  3.81633703,
        2.91511556, -1.24569189, -3.72716214, -2.54549857,  0.87262548])
```

```
from scipy.optimize import curve_fit

def func(x, A, w, phi):
    return A*np.cos(w*x+phi)

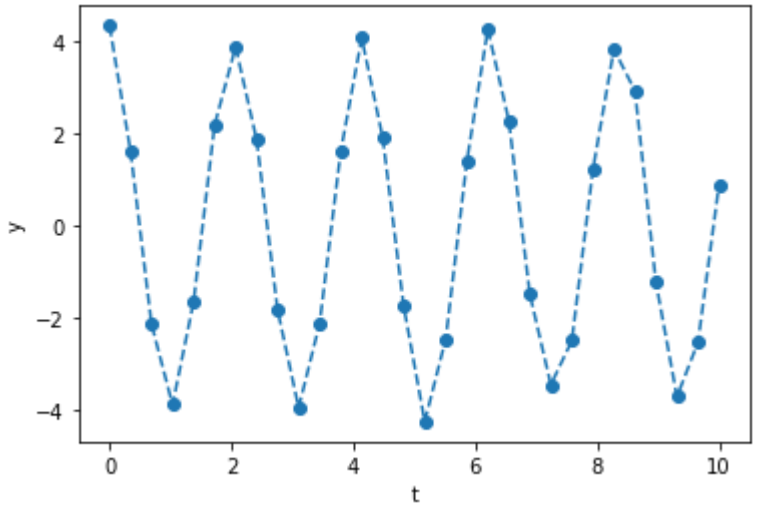
popt, pcov = curve_fit(func, t_data, y_data, p0=(4, np.pi, 0))
popt
```

```
array([3.94836219, 2.99899521, 0.10411352])
```

Initial Guess

Curve Fitting

- Curve fitting is the process of constructing a curve, or mathematical function, that has the best fit to a series of data points

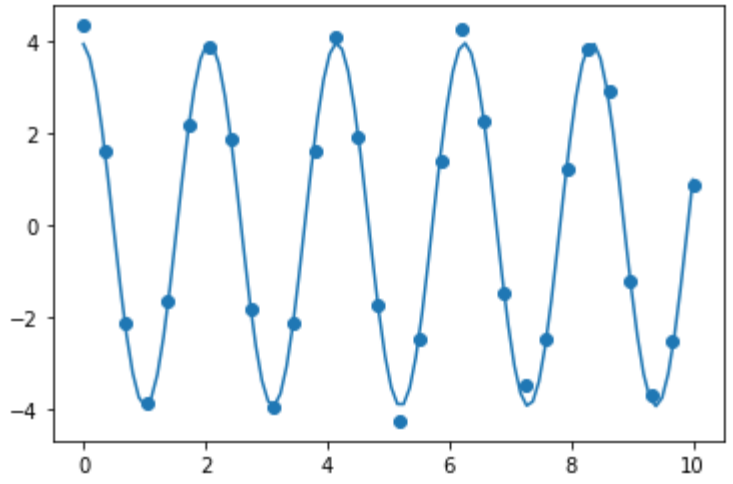


Curve Fitting

```
A, w, phi = popt
```

```
t = np.linspace(0, 10, 100)
y = func(t, A, w, phi)
```

```
plt.scatter(t_data,y_data)
plt.plot(t,y);
```



Further Resources

- https://scipy.org/
- https://www.youtube.com/watch?v=jmX4FOUEfgU&ab_channel=Mr.PSolver